CS260 Lab 5: C# Generic Extension

Objective
The purpose of this assignment is to assess your ability to design and implement C# programs that combine object-oriented design with generic programming. In each problem you will read structured data from a text file, instantiate typed objects from that data, organize the objects using a partial class, protect parts of a class hierarchy using sealed classes, add functionality to existing types through extension methods, and write generic classes and methods with constraints — specifically a base-class constraint (where T : Animal) and an interface constraint (where T : IComparable<T>) used to compute the maximum and minimum element of an array.
You must submit, for each problem, the complete original source code (all .cs files and the data file) plus a screenshot showing the program running and producing the expected output. You are expected to write all code yourself; the use of AI code-generation tools (e.g., ChatGPT, GitHub Copilot, or similar) is not allowed, and any submitted code must be your own work.

Total: 25 points

Problem — Animal Shelter Registry (25 points)
Build a console application that loads animals from a text file, stores them in a shelter, prints a report, and identifies the oldest and youngest animals.

Data file: animals.txt
Each line has the format Type,Name,Age where Type is either Dog or Cat:
Dog,Rex,5
Cat,Whiskers,3
Dog,Buddy,7
Cat,Mittens,2

Required implementation
Base class: abstract class Animal : IComparable<Animal> with properties Name (string) and Age (int), a constructor, and CompareTo(Animal other) that compares animals by Age.

Sealed classes:
sealed class Dog : Animal and
sealed class Cat : Animal.
Each overrides a Speak() method (e.g., "Woof", "Meow").

Partial class: partial class AnimalRegistry, split across two files:
AnimalRegistry.cs — LoadFromFile(string path) reads the file, parses each line, and creates the correct Dog/Cat object.
AnimalRegistry.Report.cs — reporting methods (e.g., PrintAll()).

Generic class with base-class restriction: class Shelter<T> where T : Animal and IComparable<T> .
The class has instance variables T[] array, with Add(T item), and a Count property.

Define the following two methods in the class:
– public T Max()
– public T Min()

Inside both of the methods, implement the comparison loop yourself using sequential processing. Throw an exception if the array is empty. Create an object of the Shelter class, add different types of Animal objects using the Add method. Further, by using the Max, and Min methods determine the oldest and youngest animal.

Extension methods: a static class AnimalExtensions containing at least:
public static string ToDisplayString(this Animal a) → e.g., "Rex (Dog, 5 years)"
Demonstrate how this method can be used using an Animal object.

Main: read the file, create objects, add them to the shelter object, print each animal from the array by using the extension method, print the oldest and youngest animals, and print the average age. Handle a missing file gracefully.

Expected output (example)
Loaded 4 animals.
Rex (Dog, 5 years)
Whiskers (Cat, 3 years)
Buddy (Dog, 7 years)
Mittens (Cat, 2 years)

Oldest: Buddy (Dog, 7 years)
Youngest: Mittens (Cat, 2 years)

Average age: 4.25

Rubrics

Criterion	Points	Full Credit	Partial Credit
1. File reading and object creation	4	Reads animals.txt, parses all lines, creates correct Dog/Cat objects	1–3: minor parsing or object-creation errors
2. Sealed classes and partial class	4	Dog/Cat are sealed; AnimalRegistry is partial across two files and both parts are used	1–3: missing sealed, not split, or one part unused

3. Extension methods	4	AnimalExtensions defines correct this methods and they are called in Main	1–3: wrong signature, not called, or weak output
4. Generics with restrictions	10	Shelter<T> where T : Animal and IComparable<T>works; Max/Min methods are written manually without LINQ, and return correct oldest/youngest	1–9: missing constraint, LINQ used, or incorrect result
5. Program, output, and submission	3	Compiles, output is correct, screenshot + all source/data files submitted	1–2: minor compile/output issue or missing one submission item
Total	25		

Submission Requirements
Original program: all .cs files (including both files of the partial class), and the data file (animals.txt), in a .zip file.
Screenshot: a clear screenshot of the console window showing the program running and producing the expected output. The screenshot must show your program's own output (not an editor or IDE output pane only).

Notes
Do not use LINQ's Max()/Min() for the generic algorithm requirement — implement the comparison loop yourself.
Extension methods must be used (called) in Main, not just defined.
AI-generated code is not permitted; submissions may be checked and/or defended orally.